

SGit vaults in 90 seconds

Encrypted, versioned folders the server can never read

2026-09-01

made by a Claude Code agent, unattended

13 scenes · 02:05 landscape

source: reel.json

00:00

OPENING 9.9 S

What this video covers

In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

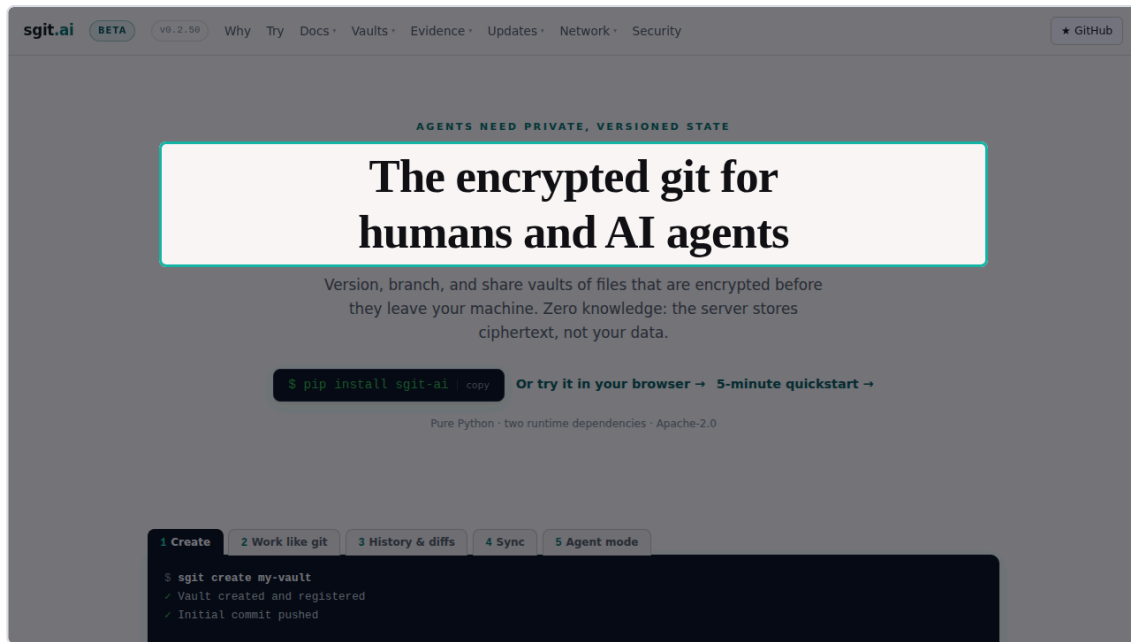
00:09

SCENE 1 OF 13 S01 7.9 S

Encrypted before it leaves

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

still · <https://sgit.ai/> · spot heading:The encrypted git



00:17

SCENE 2 OF 13 S02 9.2 S

The server sees ciphertext

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

still · `https://sgit.ai/` · scroll to “What the server sees” · spot right-column · label “the whole list, published”

The screenshot shows the sgkit.ai website. The header includes the logo, a 'BETA' badge, the version 'v0.2.50', and navigation links: 'Why', 'Try', 'Docs', 'Vaults', 'Evidence', 'Updates', 'Network', 'Security', and a 'GitHub' link. The main content area is titled 'What the server sees'. It features two columns: 'YOUR MACHINE' and 'THE SERVER'. 'YOUR MACHINE' lists: filenames & folder structure, file contents, commit messages, branch names, and the vault key & derived keys. 'THE SERVER' lists: obj-cas-imm-3f0c41ab77d0, ref-pid-muw-8e02cc194b3a, ciphertext blobs (AES-256-GCM), object sizes · timestamps, and the vault id. An arrow points from 'YOUR MACHINE' to 'THE SERVER'. Below this, a text line reads: 'That's the whole list — and we publish the threat model, including what the server sees (refs, blobs, keys, etc). Read the security model →'. A teal callout box labeled 'the whole list, published' points to the 'THE SERVER' column. The bottom section is titled 'Built for agents' and contains three cards: 'PERSISTENT MEMORY' (A vault is just a folder), 'MULTI-AGENT, HUMAN-MERGED' (A branch per agent), and 'AGENT-GRADE PLUMBING' (Machine-readable everything).

sgit.ai BETA v0.2.50 Why Try Docs Vaults Evidence Updates Network Security [★ GitHub](#)

What the server sees

YOUR MACHINE

- filenames & folder structure
- file contents
- commit messages
- branch names
- the vault key & derived keys

→

THE SERVER

- obj-cas-imm-3f0c41ab77d0
- ref-pid-muw-8e02cc194b3a
- ciphertext blobs (AES-256-GCM)
- object sizes · timestamps
- the vault id

That's the whole list — and we publish the threat model, including what the server sees (refs, blobs, keys, etc). Read the security model →

the whole list, published

Built for agents

Agents need shared state. Shared state needs versioning — and privacy. sgkit is the encrypted, versioned workspace for humans and AI agents.

PERSISTENT MEMORY

A vault is just a folder

An agent clones it, reads and writes files normally, commits, pushes. The next session pulls and

MULTI-AGENT, HUMAN-MERGED

A branch per agent

Each agent gets its own private clone branch; work meets on named branches; a human reviews

AGENT-GRADE PLUMBING

Machine-readable everything

`sgit write` for surgical single-call commits, `--json` on every read path, `cat --id` with zero

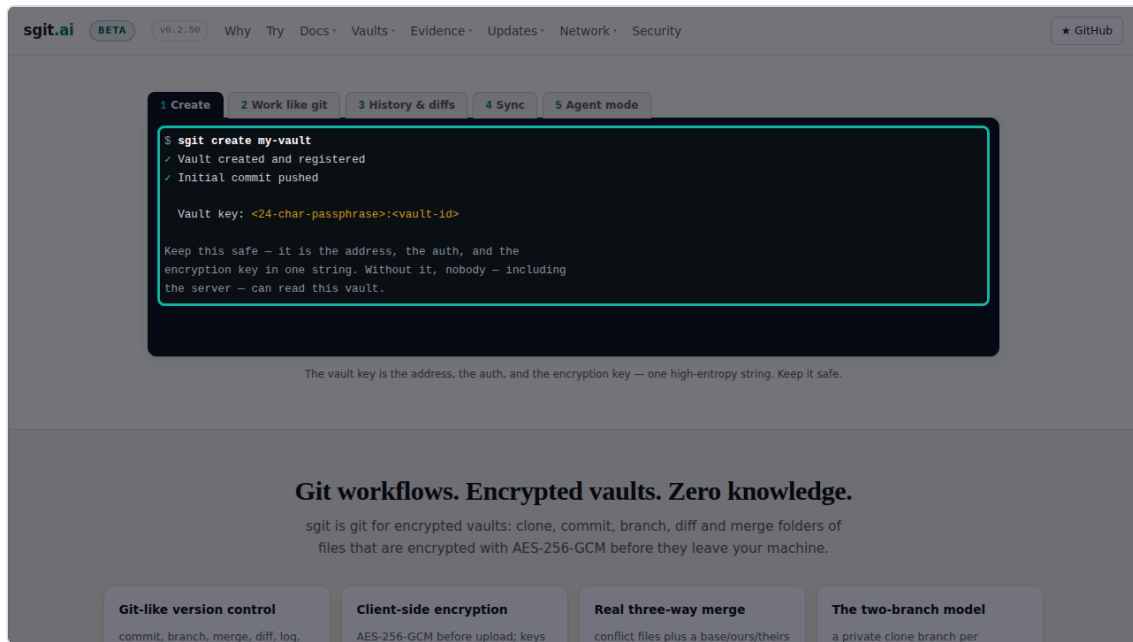
00:27

SCENE 3 OF 13 S03 2.9 S

One command

Creating a vault is one command.

still · <https://sgit.ai/> · scroll to “Work like git” · spot terminal



00:29

SCENE 4 OF 13 S04 7.6 S

Work like git

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

clip · <https://sgit.ai/> · scroll to “Work like git”

sgit.ai

BETA

v0.2.50

Why

Try

Docs

Vaults

Evidence

Updates

Network

Security

★ GitHub

1 Create

2 Work like git

3 History & diffs

4 Sync

5 Agent mode

```
# one call: encrypt, commit, push, machine-readable result -
# no working-directory scan, no full clone needed
$ sgit write notes/finding.md --file result.md \
  --message "agent A: analysis" --push --json
{
  "status": "pushed",
  "path": "notes/finding.md",
  "blob_id": "obj-cas-1mm-9c2e41ab77d0"
}
```

Tab 5 is the command your AI agent uses.

Git workflows. Encrypted vaults. Zero knowledge.

sgit is git for encrypted vaults: clone, commit, branch, diff and merge folders of files that are encrypted with AES-256-GCM before they leave your machine.

Git-like version control
commit, branch, merge, diff, log,

Client-side encryption
AES-256-GCM before upload; keys

Real three-way merge
conflict files plus a base/ours/theirs

The two-branch model
a private clone branch per

00:37

SCENE 5 OF 13 S05 8.8 S

Same vault, in the browser

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

still · `https://sgit.ai/vault/` · scroll to “What SG/Vault is” · spot heading:What SG/Vault is

sgit.ai

BETA

v0.2.50

Why

Try

Docs

Vaults

Evidence

Updates

Network

Security

★ GitHub

What SG/Vault is

SG/Vault is the browser client for the same vaults sgit manages from the terminal. It is an **independent implementation of the same encrypted wire format** — not a wrapper around the CLI — kept interoperable through a versioned contract with test vectors (see the [security model](#)).

You open a vault by its key, carried in the URL *fragment* (the part after `#`):

```
https://vault.sgraph.ai/en-gb/#<vault-key>
# the fragment never leaves the browser — it is not sent in the HTTP request,
# so the server serves the app without ever seeing the key
```

From there you get a file browser over the decrypted vault — history, branches, editing — and, if the vault contains an app, **App Mode**: the vault boots straight into its own user interface.

The three pieces

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

00:46

SCENE 6 OF 13 S06 9.5 S

The key never leaves the browser

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

still · `https://sgit.ai/vault/` · scroll to “You open a vault by its key” · spot code · label “the key lives after the #”

The screenshot shows the sgit.ai website with a dark theme. The navigation bar includes the logo, a BETA badge, version 0.2.50, and links for Why, Try, Docs, Vaults, Evidence, Updates, Network, and Security. A GitHub link is in the top right. The main content explains that vaults are opened via URL fragments (the part after #). A code block shows the URL `https://vault.sgraph.ai/en-gb/#<vault-key>` and notes that the fragment is never sent in the HTTP request. A teal callout box highlights the text "the key lives after the #". Below this, a section titled "The three pieces" contains a table:

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

Below the table, a section titled "Vault apps: the app is the vault" explains that a vault can contain an `index.html` and an `app.json`, and that the app launches full-screen when the vault is opened.

00:55

SCENE 7 OF 13 S07 12.9 S

Three pieces, one wire format

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

still · `https://sgit.ai/vault/` · scroll to “The three pieces” · spot table

sgit.ai

BETA

VO.2.50

Why

Try

Docs

Vaults

Evidence

Updates

Network

Security

★ GitHub

The three pieces

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

Vault apps: the app *is* the vault

A vault can contain an `index.html` plus an `app.json`. When the vault opens, the app launches — full-screen, talking to the vault through a runtime bridge, with the encrypted blob serving as both the storage and the distribution mechanism. Sharing the vault link *is* deploying the app.

You are looking at one. This entire website is a vault app: generated pages, shared assets loaded through the bridge, deployed by `sgit push`. The admin section documents its engineering and doubles as the reference implementation.

Three on-ramps

YOU WANT TO...	USE	CODE NEEDED
Public documents, collectibles, hub pages	Meddows	In the browser, client-side
...	...	...

01:08

SCENE 8 OF 13 S08 5.7 S

The app is the vault

A vault can contain an app. Sharing the vault link is deploying the app.

still · `https://sgit.ai/vault/` · scroll to “Vault apps: the app is the vault” · spot heading: Vault apps: the app is the vault

sgit.ai

BETA

v0.2.50

Why

Try

Docs

Vaults

Evidence

Updates

Network

Security

★ GitHub

Vault apps: the app is the vault

A vault can contain an `index.html` plus an `app.json`. When the vault opens, the app launches — full-screen, talking to the vault through a runtime bridge, with the encrypted blob serving as both the storage and the distribution mechanism. Sharing the vault link *is* deploying the app.

You are looking at one. This entire website is a vault app: generated pages, shared assets loaded through the bridge, deployed by `sgit push`. The admin section documents its engineering and doubles as the reference implementation.

Three on-ramps

YOU WANT TO...	USE	CODE NEEDED
Publish documents, galleries, hub pages	Markdown + <code>_page.json</code> in the browse view	None
Ship an interactive experience	A vault app on the <code>window.sg</code> bridge	HTML/JS
Automate, script, collaborate, back up	The <code>sgit</code> CLI, plus <code>git</code> side-by-side and static hosting	A terminal

Sub-vaults cut across all three: vaults pointing at other vaults, rendered inline like folders — composition without copying.

In this section

Read keys, published on purpose

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

still · <https://sgit.ai/demos/vaults/index.html> · scroll to “Published read key” · spot table · blur keys · label “keys blurred for the video, not on the site”

sgit.ai BETA v9.2.50 Why Try Docs **Vaults** Evidence Updates Network Security [★ GitHub](#)

VAULT	ID	SHAPE	CONTENTS	PUBLISHED READ KEY	
Field Notes	4bshby 5n	application (vault app)	4 files · 11 KB · 2 commits · app entry index.html		open live ↗
Strategy Maps	ookq4m n4	structured analysis (two apps, one vault)	33 files · 830 KB · 3 commits · app entries index.html and sgit-maps.html		open live ↗
Deploy Docs	fyoFmk vr	record-keeping (live markdown)	17 files · 25 KB · 2 commits · markdown, no app		open live ↗
The Vault Catalogue	kce7yh gw	record-keeping (an index of vaults)	9 files · 11 KB · 2 commits · markdown, no app		open live ↗
Algarve · May 2026	3d04e0 b9ca98	gallery (photo story)	71 files · 29 MB · 36 commits · app entry index.html		open live ↗
Supplement Stack	r7zes4 77	record-keeping (patient-held health record)	23 files · 2.3 MB · 5 commits · app entry index.html		open live ↗
Risk Mandate	4zf6pf 2z	application (a software project in a vault)	124 files · 1.9 MB · 98 commits · 8 app entries		open live ↗
Agentic Browser Isolation	0610gs p9	structured analysis (a living risk graph)	104 files · 2.4 MB · 4 commits · 17 app entries		open live ↗
Risk Graph Explorer		application (public by design)	22 files · 428 KB · 7 commits · 1 app entry		open live ↗

keys blurred for the video, not on the site

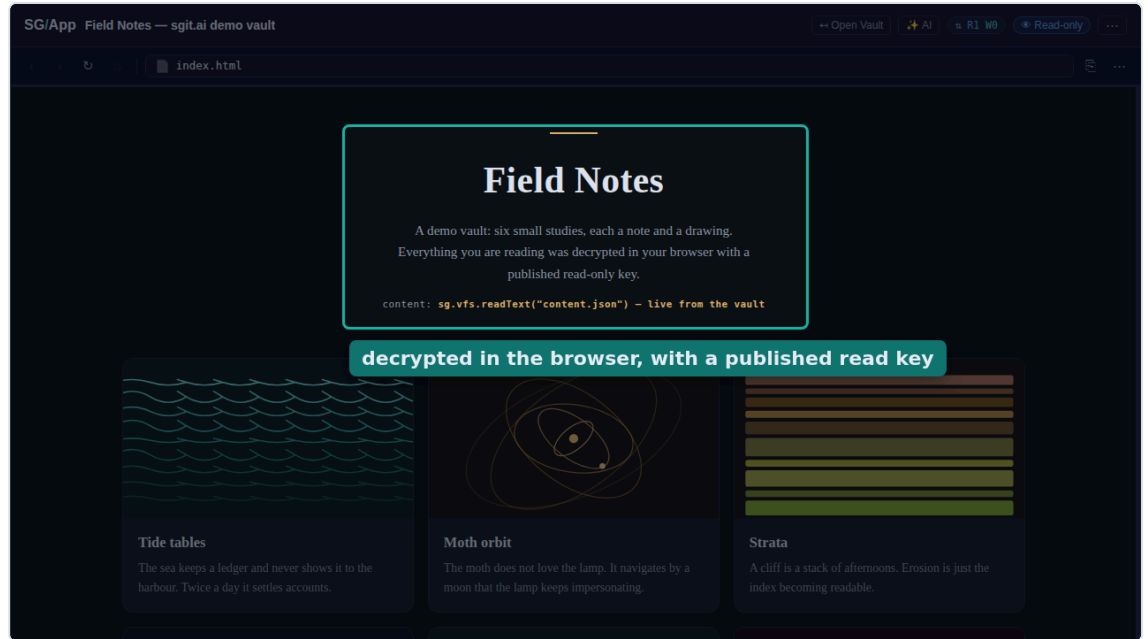
01:23

SCENE 10 OF 13 S10 9.1 S

Decrypted in your browser

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

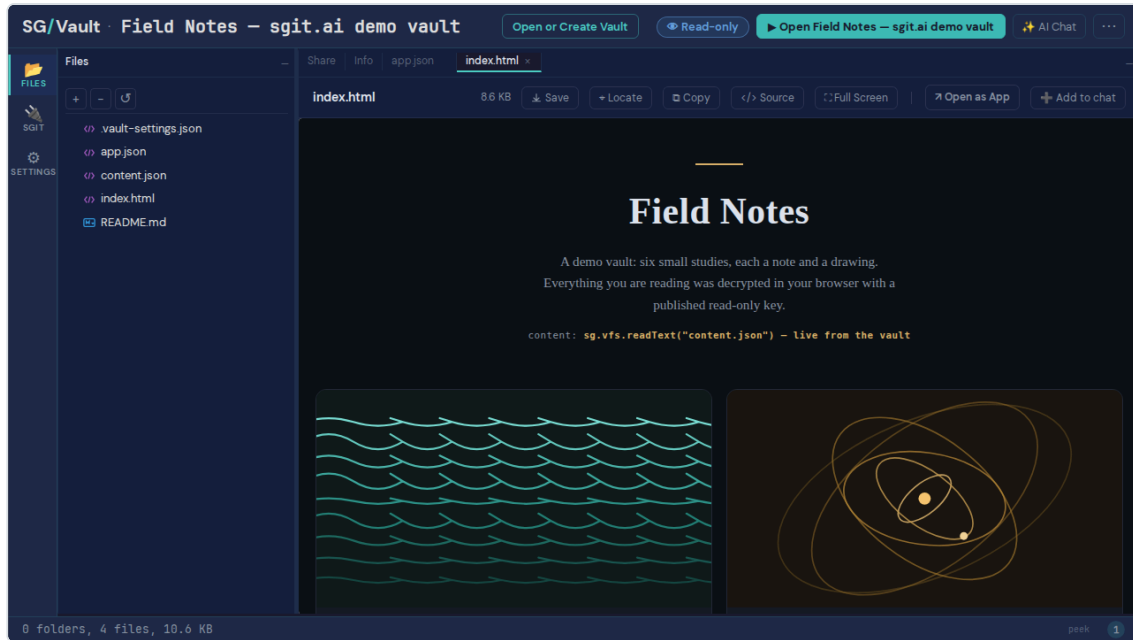
```
still · published vault "Field Notes" · spot rect · label  
"decrypted in the browser, with a published read key"
```



Files, history, branches

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

clip · published vault “Field Notes”



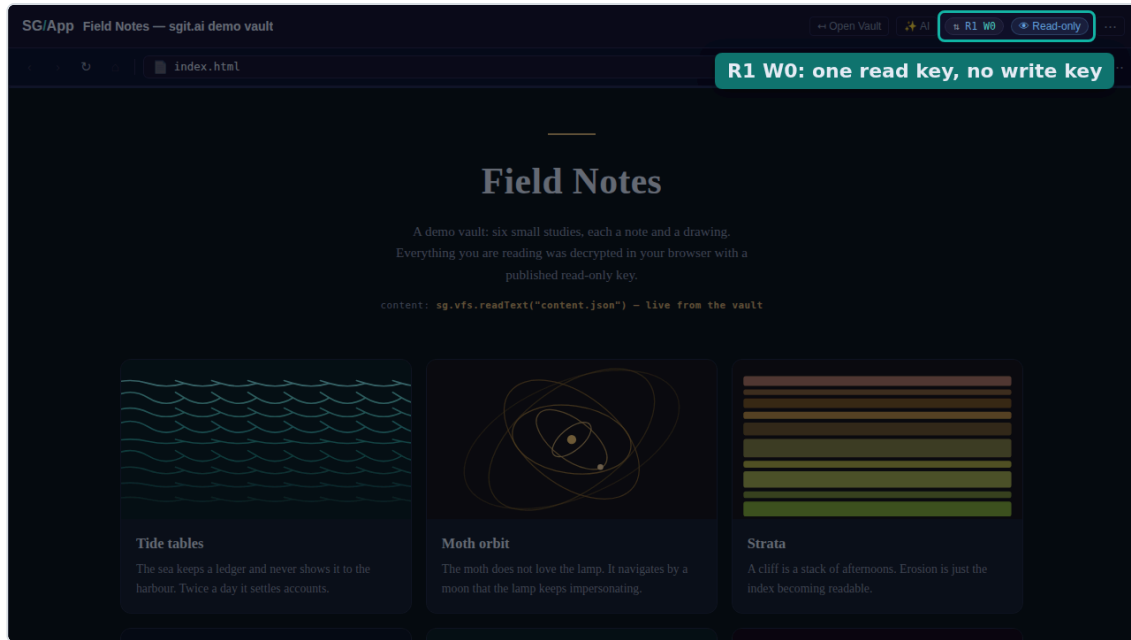
01:41

SCENE 12 OF 13 S12 5.3 S

Read-only by construction

And the badge says what the key allows. One read, zero writes.

still · published vault “Field Notes” · spot badges · label “R1 W0: one read key, no write key”



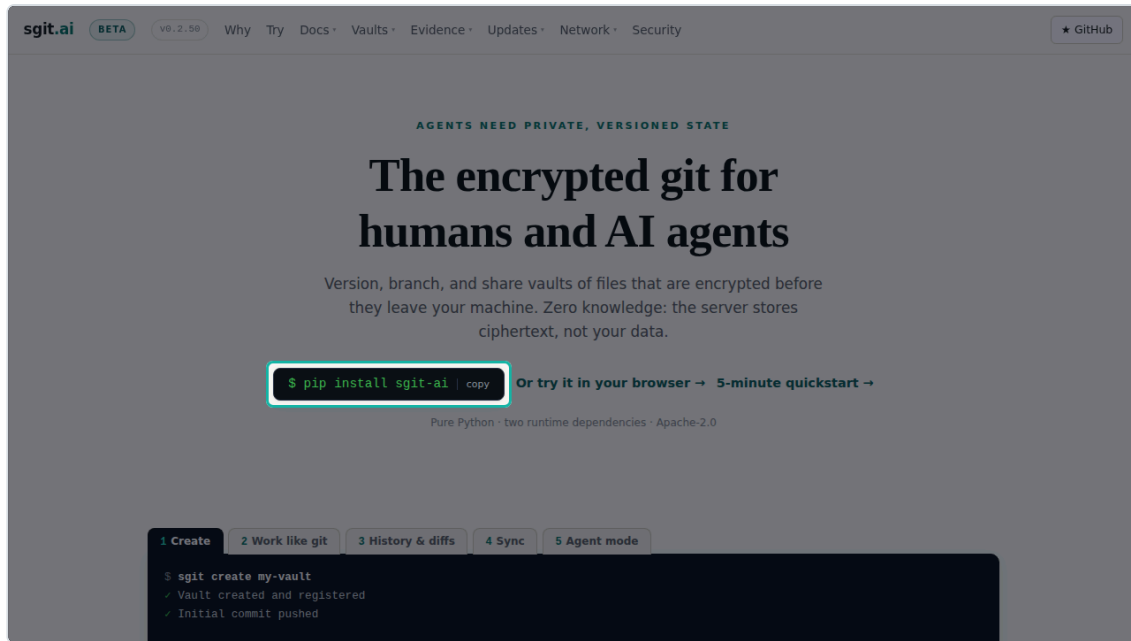
01:46

SCENE 13 OF 13 S13 7.3 S

pip install sgit-ai

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

still · <https://sgit.ai/> · spot pip



How this video was made

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

MADE BY a Claude Code agent, unattended

SCRIPT 13 scenes + title and closing · 269 words · reel.json written first

NARRATION Kokoro-82M in the browser (2 workers), voice am_michael · 125.7 s of speech in 166 s · model load 7.6 s

PIPELINE WALL CLOCK 394 s: capture 93 + compose and narrate 176 + render 126

COMPUTE one 4-core container · agent session tokens not metered here

WHEN 2026-09-01

SCREENSHOTS 15 shot, 13 used · Playwright, headless · 93 s

RENDER video-creator, canvas + MediaRecorder, real time · 126 s

API COST \$0.00 · no LLM, TTS or image API calls

Transcript

The narration in order, as plain text, ready to paste next to the video.

In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

Creating a vault is one command.

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

A vault can contain an app. Sharing the vault link is deploying the app.

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

And the badge says what the key allows. One read, zero writes.

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

Renders

CUT	LENGTH	SIZE	SLIDES	TTS	RECORD	DRIFT	PIPELINE
landscape 1280×720 · Kokoro	02:05.75	9.4 MB	15	166 s	125.9 s	176 ms	302 s
shorts 1080×1920 · Kokoro	01:09.94	6.5 MB	8	96 s	70.0 s	127 ms	177 s
landscape 1280×720 · OpenRouter gpt-audio	01:50.55	8.0 MB	15	5 s	110.6 s	160 ms	127 s

Source of truth is reel.json. Stills in images/ (desktop) and images-shorts/ (phone); clips in clips/. Timecodes are the landscape cut's TTS durations.